```
var_dump(strpos($quote,'and', 10));
```

```
Output:
int(41)
```

Remember that the `str*pos()` functions are returning integer values indicating which byte in the buffer the sub-string starts at, which is fine for single-byte encoded ASCII text but less useful for multi-byte encodings where it is generally more intuitive to work with character positions. The `mb_str*pos()` functions work at the character level and with a range of multi-byte encoding formats (UTF-8 being the default for PHP and the most ubiquitous on today's internet). Here's a short demonstration; let's start with some multi-byte data:

```
$quote = "\u{15e}\u{fc}phe ve belirsizlik i\u{e7}ermeyen kat\u{131}
kurallar talep ediyoruz!";
print $quote . PHP_EOL;
```

```
Output:
Şüphe ve belirsizlik içermeyen katı kurallar talep ediyoruz!
```

We've translated `$quote` to Turkish (many thanks to my good friend Esra for providing the translation); the non-ASCII characters are expressed as Unicode escapes, and the result is a string that is mostly single-byte ASCII with some multi-byte characters. If you are using the provided Docker image for the book, then the default encoding will be UTF-8, as we can see if we obtain the byte values for the first five characters.

```
var_dump(bin2hex("\u{15e}\u{fc}phe"));
```

```
Output:
string(14) "c59ec3bc706865"
```

The byte sequence is *0xC5 0x9E* (Latin Capital S with Cedilla – Ş), *0xC3 0xBC* (Latin Small U with Diaresis – ü), then *0x70 0x68 0x65* (Latin Small P, H, and E, all in the ASCII range). Let's compare `strpos()` with `mb_strpos()`:

```
var_dump(strpos($quote, 'phe ve'));
var_dump(mb_strpos($quote, 'phe ve'));
```